Simple Beats Clever

cost, performance and complexity trade-offs

Watch the 3-min video

In one line: every component costs money, operational effort and new failure modes; design for your real scale plus reasonable growth, compare designs by total cost of ownership, and keep the simplest one that meets the numbers.

1 Every box has a price

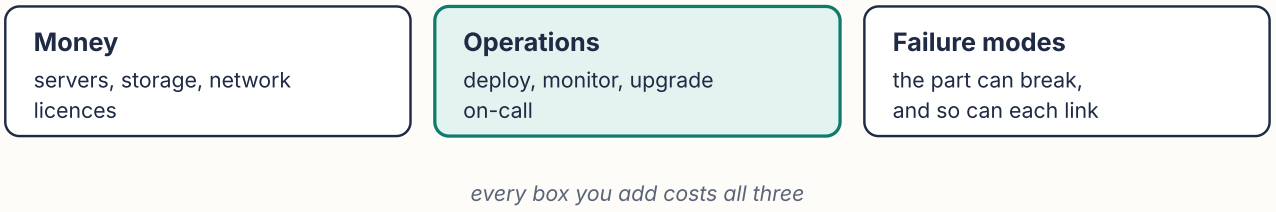


Figure 1. The cloud bill is the smallest part of what a component costs.

The real cost is the **total cost of ownership (TCO)**: infrastructure plus engineering time, on-call pain and slower feature work. A design that saves 10% on servers but needs two more engineers to run it is not cheaper.

2 A worked example

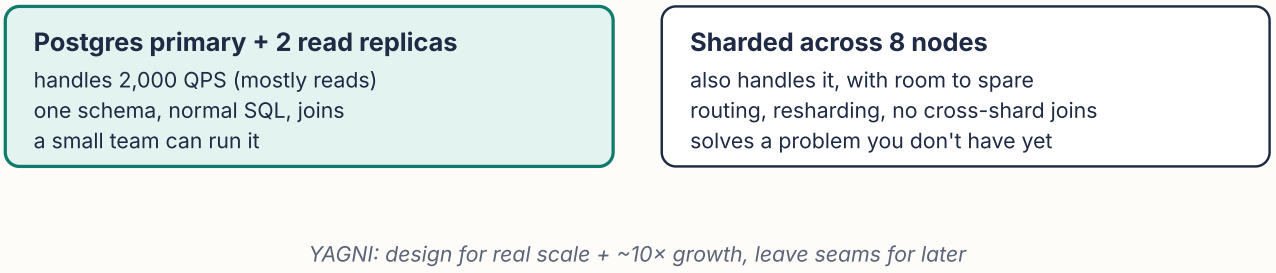


Figure 2. Both designs meet the load; the simple one does it with far less to build and run.

3 YAGNI: you aren't gonna need it

Design for your real scale plus reasonable growth, perhaps 10×, not 100×. Leave clear **seams** (a data-access layer, an interface in front of the store) so you can add caching or sharding later, when the numbers demand it.

TRIGGERS TO ADD COMPLEXITY

Add sharding when the primary's write load or data size nears its limits even after scaling up; add a cache when read latency or database load misses targets; split a service when a team or a scaling need demands it. Each trigger is a number, not a feeling.

4 Estimating cost per request

WORKED EXAMPLE

Load: 2,000 queries/s × ~2.6 M seconds a month ≈ **5.2 billion requests a month**.
Monthly cost: add up servers, storage, network and the share of engineering time.
Divide: say $6,000 / 5,200 million ≈ **$1.15 per million requests**. Now two designs can be compared with real numbers instead of opinions.

5 Performance has a price curve

Going from 500 ms to 100 ms may be cheap: an index, a cache, fewer round trips. Going from 10 ms to 2 ms can cost a fortune: more memory, more regions, specialised hardware. Spend where users actually feel the difference, and stop when the requirement is met.

COMMON MISTAKES

- Big-tech architecture on day one: microservices, Kafka, sharding for 1,000 users.
- Comparing designs on cloud cost only.
- Optimising latency far beyond what users notice.
- No seams, so the simple design can't grow later.

6 Key terms

TCO: total cost: infrastructure + people + risk

Seam: an interface that lets you swap parts later

Operational load: work to deploy, monitor, fix

YAGNI: don't build what you don't need yet

Cost per request: monthly cost ÷ monthly requests

Over-engineering: complexity beyond the requirements

7 Interview questions

Q1. When is a simpler design better, even if it scales less? BASIC
When it meets the real load with headroom: it's cheaper to run, easier to operate, and has fewer failure modes.

Q2. How do you estimate the cost of a design? BASIC
Estimate the load, add up monthly infrastructure and people costs, and compute cost per request or per million requests.

Q3. What is YAGNI in system design? BASIC
Don't build for scale or features you don't need yet; design for real scale plus ~10× growth, and leave seams to extend later.

Q4. 2,000 QPS, mostly reads. Shard or not? SCENARIO
Not yet: a Postgres primary with read replicas (and maybe a cache) handles it comfortably. Shard when write load or data size approaches a single primary's limits.

Q5. Your team proposes Kafka for 50 events a second. Your view? SCENARIO
Probably overkill: a database table or a managed queue handles 50/s with far less to operate. Kafka pays off at high throughput, replay or many consumers.

Q6. Product wants p99 from 40 ms to 10 ms. How do you decide? SCENARIO

Ask what users or revenue gain, estimate the cost (memory, regions, rework), and compare. If 40 ms already feels instant, spend elsewhere.

Q7. Isn't designing for 100× scale the safe choice? TRICKY

No. It costs money and complexity now, slows feature work, and the real bottlenecks at 100× are hard to predict. Seams plus measured triggers are safer.

Q8. Can a simpler design be less reliable? TRICKY

Sometimes; a single database is a SPOF. Simple doesn't mean no redundancy: keep replicas and failover, just avoid complexity the requirements don't ask for.

Next: SD-A-019 · Stateless vs Stateful backendroadmap.com